

Date: _____

JDBC in Java

□ M □ T □ W □ T □ F □ S □ S

Java Database Connectivity :->

Java Database Connectivity (JDBC) is an API that helps applications to communication with databases, it allows Java program to connect to a database, run queries, retrieve, and manipulate data.

Because of JDBC, Java applications can easily work with different relational databases like MySQL, Oracle, PostgreSQL, etc.

JDBC Driver ->

JDBC drivers are software components that provided the necessary functionality to connect Java applications to different types of databases.

4 Types of JDBC Driver

① Type 1 -> JDBC-ODBC Bridge.

② Type 2 -> Native-API Driver

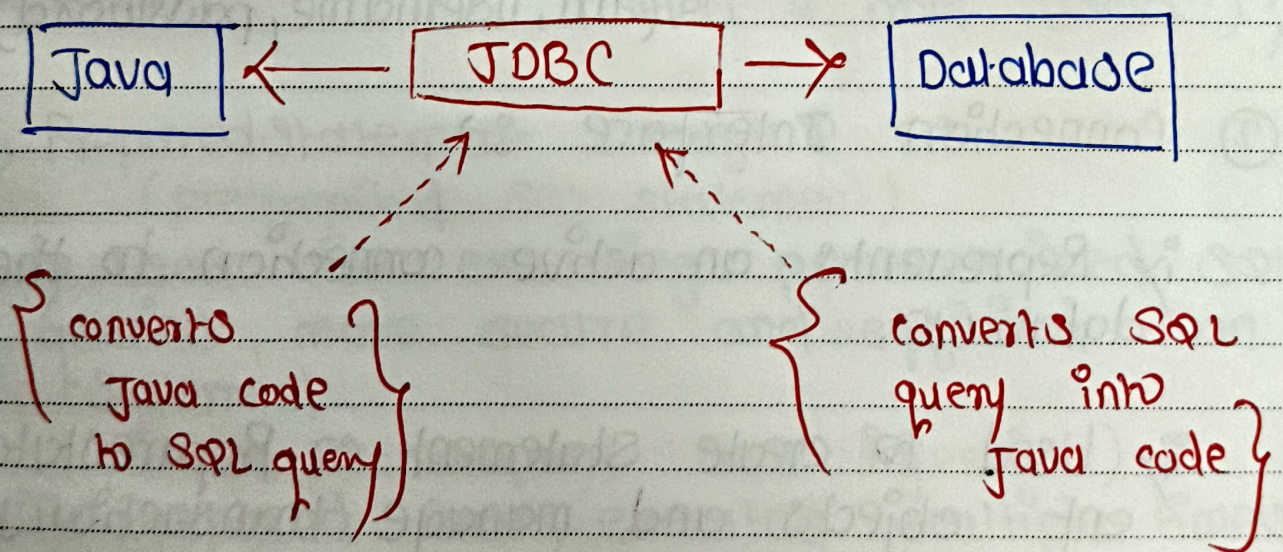
③ Type 3 -> Network Protocol Driver

④ Type 4 -> Thin Driver (Pure Java)

most commonly used.

** The older drivers like JDBC-ODBC bridge driver has been deprecated and no longer supported in modern java versions.

The Type 4 - Thin Driver is latest Driver so it is used most oftenly rather than other drivers because of they are outdated.



Date: _____

□ M □ T □ W □ T □ F □ S □ S

* JDBC components :->

① Driver Manager Class :->

- i) Manages a list of database drivers and establishes connections to databases.
- ii) Calls the appropriate driver when `getConnection()` is used.

Example ->

```
Connection con = DriverManager.getConnection(
    (url, username, password);
```

② Connection Interface :->

- i) Represents an active connection to the database.
- ii) Used to create Statement or PreparedStatement objects and manage transactions.

Example ->

```
Connection con = DriverManager.getConnection(
    url, username,
    password);
```


③ Statement Interface :-

i) Used to execute static SQL queries (without parameters).

ii) Types → executeQuery() → for select
executeUpdate() → for insert,
update, delete

Example →

```
Statement stmt = con.createStatement();
```

```
ResultSet rs = stmt.executeQuery("select * from users");
```

④ PreparedStatement Interface :-

(precompiled SQL statement)

i) Used for executing parameterized SQL queries, more secure and efficient than Statement.

(helps to prevent SQL injection)

ii) Prepared Once, executed multiple times with different values.

Example →

```
PreparedStatement ps = con.prepareStatement("select * from user where id = ?");
```

```
ps.setInt(1, 101);
```

```
ResultSet rs = ps.executeQuery();
```


Date: _____

□ M □ T □ W □ T □ F □ S □ S

Interface

⑤ Callable Statement :→

is used to call stored procedures in the database.

Example
(SQL)



delimiter #

```
create procedure getUserById  
    (in userId int)
```

```
begin
```

```
    select * from users where id =  
        userId;
```

```
end #
```

```
delimiter ;
```

Example
(Java)



```
CallableStatement cs = con.prepareStatement  
    ("{call getUserById(?)}")
```

```
cs.setInt(1, 101);
```

```
ResultSet rs = cs.executeQuery();
```

↳ A stored procedure is like a function, it's a predefined / prewritten SQL program.

⑥ ResultSet Interface :-

- i) stores the result of select queried.
- ii) It acts like a table of data that you can iterate over to retrieve each row one at a time.

Note →

- Returned by `executeQuery()` method
- Used to read data from database.
- provides methods to navigate and extract data from each row.
- Allows reading row by row using `rs.next()`.

Cursor in ResultSet →

- Initially, the cursor is before the first row.
- You use `rs.next()` to move to the next row.
- When there are no more rows, `rs.next()` returns false.

Example of ResultSet →

```
Statement stmt = con.createStatement();
```

```
ResultSet rs = stmt.executeQuery("select-  
id, name from users");
```

```
while (rs.next()) {
```

```
    int id = rs.getInt("id");
```

```
    String name = rs.getString("name");
```

```
    System.out.println(id + " " + name);
```

```
}
```

⑦ SQL Exception ⇒

i) Handled exceptions during JDBC operations.

ii) getMessage(), getErrorCode(), getSQLState().

Example. →

```
catch (SQLException e) {
```

```
    System.out.println("Error: " +  
e.getMessage());
```

```
}
```


8) close() in JDBC

The close() method is used to release JDBC resources (like connection, statement, ResultSet)

After they are no longer needed close() helps to close all this resource and helps us to avoid memory leaks

What we need to close →

ResultSet → rs.close()

Statement → stmt.close()

Connection → con.close()

PreparedStatement → ps.close()

Callable Statement → cs.close()

Not closing the connection, statement, ResultSet can lead to memory leaks and database crashes

Date: _____

□ M □ T □ W □ T □ F □ S □ S

JDBC code :- ~~→~~

```
import java.sql.*;
```

```
public class JDBCExample {
```

```
    private static final String url =
```

```
        "jdbc:mysql://localhost:3306/db";
```

```
    private static final String user =
```

```
        "root";
```

```
    private static final String password =
```

```
        "Ankit2631";
```

```
    public static void main(String[] args) {
```

```
        try { // Step ①
```

```
            // load and register the driver
```

```
            Class.forName("com.mysql.cj.jdbc.Driver");
```

// This is not compulsory or not strictly required in modern version of JDBC (Java 6 and later)

```
        // Step ② : Establish a connection.
```

```
        Connection connection = DriverManager
```

```
            getConnection(url, user, password);
```

```
        System.out.println("Connected to database");
```


// step ③ : Define the SQL query to be executed
String query = "select id, name, from student;

// step ④ : Create PreparedStatement object
PreparedStatement statement = connection.
prepareStatement(query);

// step ⑤ : Execute the query and get result
ResultSet resultSet = statement.executeQuery();

// step ⑥ : Process the result set

while (resultSet.next()) {

int id = resultSet.getInt("id");

String name = resultSet.getString("name");

System.out.println("id : " + id + " name " + name);

}

// step ⑦ : Close ResultSet.

if (resultSet != null) {

resultSet.close();

System.out.println("ResultSet closed");

}

Date: _____

OM OT OW OT OF OS OS

// step ⑧ : Close Statement

```
if (statement != null) {
```

```
    statement.close();  
    System.out.println("Statement closed");  
}
```

// step ⑨ : Close Connection

```
if (connection != null) {
```

```
    connection.close();  
    System.out.println("Connection closed");  
}
```

```
}  
catch (ClassNotFoundException e) {
```

```
    // Handle case where JDBC driver class  
    is not found.
```

```
    System.out.println("Driver not found: " +  
        e.getMessage());
```

```
} catch (SQLException e) {
```

```
    // Handle SQL related exceptions.
```

```
    System.out.println("SQL Error" + e.getMessage());  
}
```

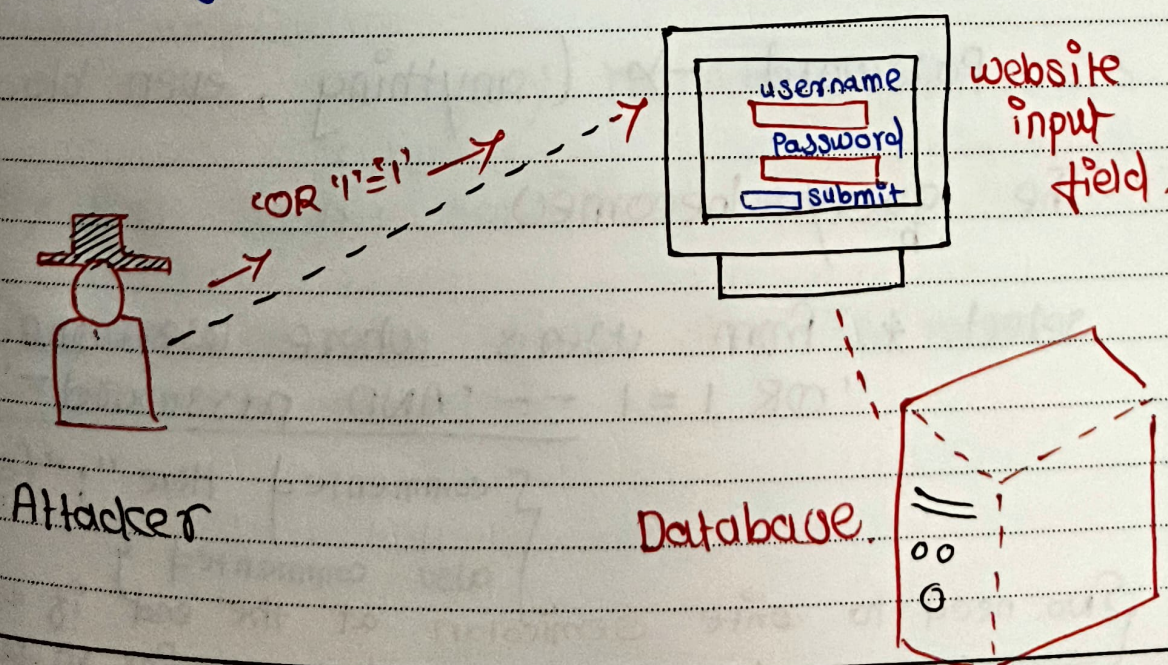
```
}  
}
```


SQL Injection :->

SQL injection is a technique used by attackers to manipulate a web application's database queries.

By injecting malicious SQL code into input fields, attackers can execute unauthorized commands and potentially gain access to sensitive information.

SQL injection attacks can also be used to delete data, manipulate records, or retrieve information from multiple databases & tables using techniques like `OR 1=1` - classic SQL injection, error-based injection, union based injection, inferential SQL injection.



Date: _____

OM OT OW OT OF OS OS

In SQL, $1=1$ is a condition that always evaluates to TRUE

When used with OR, it can alter the logic of a SQL query in dangerous ways.

Example of vulnerability $\Rightarrow$

Suppose a login system with this SQL query $\rightarrow$

```
select * from users where username = '$username'
and password = '$password';
```

while login, If the attacker input the following

username $\rightarrow$ 'OR 1=1 --

password $\rightarrow$ (anything, even black).

The query becomes.

```
select * from users where username =
"OR 1=1 -- 'AND password = '";
```

} commented Here "; (semi colon)
also commented }

{ No need to write semicolon at the end if we send one complete SQL command as a string to database }

Explanation →

i/ OR 1=1 makes the condition always true

ii/ -- is a comment in SQL, which ignores the rest of the query.

iii/ So the query is now just:

```
select * from users where username =
' ' OR 1=1
```

(As a result, the attacker is logged in without valid credentials).

* Purpose of OR 1=1 in SQL injection.

i/ Bypasses authentication.

ii/ Forces the query to return all rows.

iii/ Can reveal sensitive data.

iv/ Can be the first step in more serious attacks.

* Prevention of SQL Injection. →

To prevent SQL injection in Java with JDBC.

the best approach is to use Prepared Statement (parameterized queries).

Example →

```
String username = request.getParameter("username");
```

```
String password = request.getParameter("password");
```

```
String query = "select * from users where  
username = ? and password = ?";
```

```
PreparedStatement pstate = connection.prepareStatement(query);
```

```
pstate.setString(1, username);
```

```
pstate.setString(2, password);
```

```
ResultSet rs = pstate.executeQuery();
```

```
if(rs.next()) {
```

```
// login successful
```

```
} else {
```

```
// login failed
```

```
}
```


Note :->

⇒ Input is never directly added to the SQL string.

① without PreparedStatement normal query

```
String sql = "select * from users where  
username = " + user + "';
```

② with PreparedStatement

```
String sql = "select * from users where  
username = ?";  
pstate.set-String(1, user);
```

⇒ If user types something hicky like 'OR' it is just text.